

# A inteligência do inimigo

O inimigo para de fazer sempre a mesma coisa e passa a escolher. Você vai descobrir que escolher e fazer são duas responsabilidades diferentes — e que o contrato de ontem já servia para os dois lados.

---

**Continuação direta da aula de habilidades especiais.**

Você precisa do jogo rodando, com `habilidade_especial` e a recarga funcionando, antes de começar.

# Onde vocês pararam

---

Na aula passada o seu herói ganhou uma habilidade especial. O Guerreiro esmaga, o Mago queima, e o menu chama `jogador.habilidade_especial(inimigo)` sem perguntar qual é a classe. Você também escreveu uma recarga: o contador `_recarga_habilidade` começa em zero, sobe depois de um uso e desce dentro de `iniciar_turno`.

Foi um bom dia de POO. Mas o jogo ficou torto de um jeito que dá para perceber jogando: **o herói tem quatro opções e o inimigo tem uma.**

## A sua pasta

Continuam sendo quatro arquivos. Hoje entram dois deles, e um só na última linha.

```
rpg/  
  personagem.py    <-- entra o metodo agir  
  classes.py       <-- entra a decisao de cada inimigo  
  itens.py         nao mexa  
  main.py          <-- muda UMA linha
```

### A PROMESSA DESTA AULA

No fim do dia, o `main.py` vai ter **uma única linha diferente** da de hoje de manhã. Todo o resto do trabalho acontece dentro das classes. Se você se pegar escrevendo `if` sobre tipo de inimigo dentro do `main.py`, parou de fazer POO e voltou para o Portugal.

## O caminho

### 1 **Decidir não é agir**

Duas responsabilidades que estavam grudadas no mesmo método.

---

### 2 **Ler o próprio estado**

O Goblin descobre que está perdendo e foge da porrada.

---

### 3 **Decidir sem inventar estado**

O Orc e o Troll usam o que o jogo já sabe fazer.

---

### 4 **O contrato de ontem, do outro lado**

O inimigo ganha habilidade especial e recarga de graça.

---

### 5 **A prova**

Um inimigo novo entra sem que o combate saiba.

---

**Lembrete:** os extras do anexo B continuam opcionais. Pular todos não te atrapalha em nada.

# Sumário

---

**01** Decidir não é agir

---

**02** Ler o próprio estado

---

**03** Decidir sem inventar estado

---

**04** O contrato de ontem, do outro lado

---

**05** A prova do desenho

---

**AN** Anexo A: erros que você vai ver

---

**AN** Anexo B: extras

---

**AN** Anexo C: checklist de entrega

---

# Decidir não é agir

## O problema

Abra o seu `main.py` e procure o combate. Em algum lugar dele existe esta linha:

```
inimigo.atacar(jogador)
```

Essa linha é a inteligência inteira do seu inimigo. Ela diz: *aconteça o que acontecer, ataque*.

Repare no absurdo que isso produz. O Troll está com 3 de vida, cercado, sem chance nenhuma — e ataca. O Goblin acabou de nascer com vida cheia — e ataca. O Dragão está no último fôlego — e ataca. Nenhum deles bebe poção, nenhum se defende, nenhum recua, nenhum guarda o golpe forte para a hora certa.

### O PROBLEMA

O seu jogo tem **seis classes de inimigo** e todas se comportam igual. As diferenças entre elas são só números: o Troll bate mais forte, o Goblin tem menos vida. Trocar 40 por 100 não é criar um personagem.

E tem um agravante: o jogador tem quatro escolhas por turno, o inimigo tem zero. Combate é sobre **decisões**. Um lado que não decide não é adversário, é alvo.

## A explicação

Antes de escrever qualquer coisa, vale entender por que esse problema apareceu. Ele não é falta de código. É **duas responsabilidades diferentes ocupando o mesmo lugar**.

Olhe o método `atacar` do `Personagem`. O que ele faz? Ele rola o dado, calcula o dano, chama `receber_dano` no alvo e imprime a mensagem. Isso é **executar uma ação**. É um trabalho fechado, com começo e fim, e ele faz bem.

Agora pergunte outra coisa: quem decidiu que a ação daquele turno seria atacar?

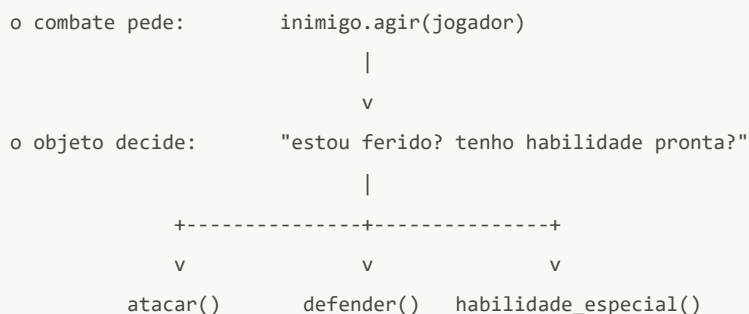
Ninguém decidiu. Está escrito na mão, lá no `main.py`. E é por isso que todo inimigo é igual: **a decisão não mora no inimigo**. Ela mora no combate, que é o único lugar onde uma decisão não pode morar, porque o combate é o mesmo para todos.

#### A IDEIA CENTRAL DE HOJE

**Escolher** uma ação e **executar** uma ação são dois trabalhos diferentes, e por isso merecem dois métodos diferentes. `atacar` executa. Um método novo, `agir`, escolhe.

Repare que isso é diferente de tudo que você fez até agora. Ontem, quando o Guerreiro sobrescreveu `habilidade_especial`, ele mudou **como** uma ação acontece. Hoje o Goblin vai sobrescrever `agir` para mudar **qual** ação acontece. O primeiro é polimorfismo sobre execução; o segundo é polimorfismo sobre decisão. Mesma ferramenta, altura diferente.

O desenho fica assim:



O combate faz uma pergunta só, sempre a mesma. O objeto responde com uma escolha que só ele sabe fazer. E as ações lá embaixo são exatamente as que já existem — você não vai escrever ação nova neste capítulo, nem no próximo.

#### A VERSÃO PADRÃO IMPORTA

O `agir` nasce na classe `Personagem`, e a versão dela simplesmente ataca. Isso não é preguiça, é o que garante que **nada quebra**: as seis classes de inimigo herdam esse comportamento no segundo em que você escreve o método, e o jogo continua funcionando exatamente como antes. Depois, uma por uma, cada classe escolhe se quer ser mais esperta.

É a mesma estratégia do `habilidade_especial` de ontem, que nasceu na mãe dizendo "não tenho habilidade". A classe base define o contrato e um comportamento seguro; as filhas decidem se sobrescrevem.

### EXEMPLO 1 · O TERMOSTATO

Um termostato tem três ações que ele sabe executar. Nenhuma delas decide nada:

```
class Termostato:
    def __init__(self, alvo):
        self.alvo = alvo
        self.ligado = None

    # as tres acoes: cada uma so executa
    def aquecer(self):
        self.ligado = "aquecedor"
        print("Aquecedor ligado")

    def resfriar(self):
        self.ligado = "ar-condicionado"
        print("Ar-condicionado ligado")

    def descansar(self):
        self.ligado = None
        print("Tudo desligado")
```

E tem **um** método que só decide, e não executa nada por conta própria:

```
def ajustar(self, temperatura):
    if temperatura < self.alvo - 2:
        self.aquecer()
    elif temperatura > self.alvo + 2:
        self.resfriar()
    else:
        self.descansar()
```

Note o que `ajustar` **não** faz: ele não liga relé, não imprime, não mexe em `self.ligado`. Ele só escolhe qual dos três chamar. Se amanhã o aquecedor mudar de tecnologia, você mexe em `aquecer` e o `ajustar` nem fica sabendo.

No RPG, `agir` ocupa exatamente a posição do `ajustar`.

## EXEMPLO 2 · O CAIXA DA LOJA

Agora com herança, que é o caso do seu jogo. Um atendente sabe atender; atendentes diferentes atendem diferente:

```
class Atendente:
    def vender(self, cliente):
        print("Vendendo para", cliente)

    def encaminhar(self, cliente):
        print("Encaminhando", cliente, "para o gerente")

    # a decisao padrao: sempre vender
    def atender(self, cliente):
        self.vender(cliente)

class Novato(Atendente):
    # o novato so decide diferente quando a compra e grande
    def atender(self, cliente):
        if cliente.valor > 5000:
            self.encaminhar(cliente)
        else:
            self.vender(cliente)
```

E a fila da loja, que é o "combate" deste exemplo:

```
for funcionario in [Atendente(), Novato(), Atendente()]:
    funcionario.atender(cliente)
```

A fila chama `atender` em todo mundo. Ela nunca pergunta quem é novato. O `Novato` não ganhou nenhuma ação nova — ele usa `vender` e `encaminhar`, que já existiam na mãe. **A única coisa que ele sobrescreveu foi o critério.**

Guarde essa frase, porque ela é o capítulo inteiro: sobrescrever a decisão não exige inventar ação nenhuma.

**Por que não chamar o método de `atacar` mesmo?** Porque aí você perde a ação. Se o `agir` do Goblin se chamasse `atacar` e às vezes defendesse, o nome estaria mentindo — e pior, você não teria mais como pedir "ataque agora" sem passar pela decisão. Repare que o Troll e o Dragão do seu `classes.py` já sobrescrevem `atacar` hoje, e isso continua certo: eles atacam de um jeito diferente. Atacar continua sendo atacar.

## EXERCÍCIO 1 · ABRIR A PORTA DA DECISÃO

ARQUIVO: PERSONAGEM.PY, DEPOIS MAIN.PY

Faça:

1. Na classe `Personagem`, crie o método `agir`, que recebe `self` e `alvo`.
2. O corpo dele tem **uma linha só**: mandar o próprio personagem atacar o alvo.
3. Não escreva `if` nenhum aqui. Esta é a decisão padrão, e a decisão padrão é sempre atacar.
4. No `main.py`, no combate, troque a chamada do ataque do inimigo por uma chamada de `agir`, passando o jogador como alvo.
5. Rode o jogo inteiro.

**Cuidado:** não mexa na chamada de `iniciar_turno` que está logo acima. Ela continua onde está e continua sendo do inimigo.

### O QUE VOCÊ DEVE VER

**Absolutamente nenhuma diferença.** O jogo roda igual, os inimigos atacam igual, as mensagens são as mesmas.

Isso é o resultado certo. Você acabou de criar o lugar onde a decisão vai morar, sem mudar nenhuma decisão ainda. Trocar a estrutura sem mudar o comportamento tem nome, e você vai ouvir esse nome muito na sua vida: **refatorar**.

**Se o Troll ou o Dragão pararam de fazer o golpe especial deles:** você escreveu o corpo do `agir` chamando `Personagem.atacar(self, alvo)` em vez de `self.atacar(alvo)`. A primeira forma força a versão da mãe e mata o polimorfismo que você tinha.

# Ler o próprio estado

## O problema

### O PROBLEMA

O Goblin é o primeiro inimigo do jogo. Ele tem 40 de vida, bate fraco, e serve para o jogador aprender o combate. Mas ele luta até o último ponto de vida com a mesma disposição do primeiro turno.

Um bicho pequeno que enfrenta um Guerreiro de armadura deveria, em algum momento, **se encolher**. Não é só sabor: um inimigo que se protege quando está morrendo obriga o jogador a pensar em quando atacar.

## A explicação

Para decidir, o objeto precisa olhar para alguma coisa. A pergunta é: olhar para o quê?

A resposta mais importante desta aula: **o objeto olha para o que ele já sabe sobre si mesmo**. O seu `Personagem` guarda vida atual, vida máxima, ataque, defesa, nível, inventário e recarga. Isso é muita informação. Praticamente toda decisão interessante sai de combinações disso, sem precisar de um único atributo novo.

### A REGRA DO DIA — LEIA DUAS VEZES

A IA de hoje **decide, mas não inventa estado**. Nenhuma classe de inimigo vai ganhar atributo novo neste capítulo nem no próximo. Se você sentiu vontade de criar um `self._contador_de_alguma_coisa`, pare e procure a informação que já existe.

Essa regra não é frescura, e ela vai fazer sentido total amanhã. Por ora, entenda o motivo prático: cada atributo novo é uma coisa a mais para inicializar, atualizar, zerar entre combates e lembrar que existe. Estado espalhado é a forma mais comum de bug em jogo.

### VIDA ABSOLUTA MENTE, VIDA RELATIVA NÃO

A primeira tentação é escrever assim:

```
if self.vida < 15:  
    self.defender()
```

Funciona para o Goblin, que tem 40 de vida. Agora imagine o mesmo critério dentro do Dragão, que tem 110. Estar com 15 de vida, para o Dragão, não é "ferido" — é "morrendo". E para um inimigo futuro com 12 de vida no total, 15 nunca acontece: ele nasce já ferido e nunca ataca.

O número que não mente é a **proporção**:

```
if self.vida < self.vida_maxima * 0.3:  
    self.defender()
```

Agora "abaixo de 30%" quer dizer a mesma coisa para todo mundo. Repare que isso só é possível porque você guardou `_vida_maxima` lá atrás, e porque criou as `@property` que deixam ler `vida` e `vida_maxima` de fora sem deixar escrever.

**Por que `self.vida` e não `self._vida` aqui dentro?** Os dois funcionam, porque você está dentro da própria classe. Prefira `self.vida`: se um dia a propriedade passar a calcular alguma coisa — vida com bônus de equipamento, por exemplo — o seu código de decisão pega a mudança de graça. Deixe o underscore para onde você realmente precisa escrever.

### EXEMPLO 1 · O CELULAR E A BATERIA

Um celular decide o modo de operação lendo o próprio nível de bateria. Ele não tem nenhum atributo chamado `modo_economia_ativado` — ele calcula a decisão na hora, a partir do que já sabe:

```
class Celular:
    def __init__(self, capacidade):
        self.capacidade = capacidade
        self.carga = capacidade

    def brilho_total(self):
        print("Tela no brilho maximo")

    def brilho_reduzido(self):
        print("Tela escurecida para poupar bateria")

    def desligar_tela(self):
        print("Tela desligada")

    def ajustar_tela(self):
        proporcao = self.carga / self.capacidade
        if proporcao < 0.05:
            self.desligar_tela()
        elif proporcao < 0.20:
            self.brilho_reduzido()
        else:
            self.brilho_total()
```

A linha `proporcao = self.carga / self.capacidade` é a mesma conta da vida do seu inimigo. E funciona igual num celular de 3000 mAh e num de 6000 mAh, exatamente como o critério de 30% funciona no Goblin e no Dragão.

Repare também na **ordem** dos testes: do caso mais grave para o menos grave. Se você inverter, o `< 0.20` pega antes e o `< 0.05` nunca roda. Esse erro vai te morder no capítulo 4.

## EXEMPLO 2 · O MESMO BICHO, DOIS CRITÉRIOS

Aqui o ponto é ver duas subclasses decidindo diferente a partir do **mesmo** estado, sem nenhum atributo novo:

```
class Animal:
    def __init__(self, energia_maxima):
        self.energia_maxima = energia_maxima
        self.energia = energia_maxima

    def cacar(self):
        print("Cacando")

    def descansar(self):
        print("Descansando")

    def viver(self):
        self.cacar()

class Lobo(Animal):
    # o lobo insiste: so descansa quase sem energia
    def viver(self):
        if self.energia < self.energia_maxima * 0.15:
            self.descansar()
        else:
            self.cacar()

class Coelho(Animal):
    # o coelho e cauteloso: descansa cedo
    def viver(self):
        if self.energia < self.energia_maxima * 0.60:
            self.descansar()
        else:
            self.cacar()
```

Lobo e Coelho leem o mesmo atributo, chamam os mesmos dois métodos, e mesmo assim são animais diferentes. **A personalidade está no número do critério**, não em código novo.

É literalmente isso que separa um Goblin covarde de um Troll teimoso.

## EXERCÍCIO 2 · O GOBLIN COVARDE

ARQUIVO: CLASSES.PY

Faça:

1. Na classe `Goblin`, sobrescreva o método `agir`, com a mesma assinatura da mãe.
2. Se a vida dele estiver abaixo de 30% da vida máxima, ele deve **se defender** em vez de atacar.
3. Em qualquer outro caso, ele ataca normalmente.
4. Escreva o critério usando proporção, nunca um número fixo de vida.
5. Acrescente uma mensagem curta antes de defender, para o jogador entender o que houve. Algo como o Goblin recuar assustado.

**Não** crie atributo novo no Goblin. **Não** escreva um `defender` novo: use o que ele herdou.

### TESTE DO EXERCÍCIO 2

```
from classes import Goblin, Guerreiro

heroi = Guerreiro("Thoric")
bicho = Goblin()

# vida cheia: deve atacar
bicho.agir(heroi)

# quase morto: deve defender
bicho._vida = 5
bicho.agir(heroi)
```

**Resultado esperado:** a primeira chamada causa dano no herói. A segunda imprime a sua mensagem de recuo e prepara a defesa, sem causar dano nenhum.

**Se as duas atacarem:** confira a conta. Com `vida_maxima` igual a 40, o limite é 12, e 5 é menor que 12. Se você escreveu `self.vida_maxima * 30` em vez de `* 0.3`, o limite virou 1200 e ele vai defender sempre.

**Se as duas defenderem:** provavelmente é o caso acima, ou você inverteu o sinal da comparação.

**Sobre mexer em `_vida` no teste:** aqui é permitido, porque isto é um teste e você está simulando uma situação. Dentro do jogo continua proibido.

## LEIA A FONTE

Documentação oficial do Python, em português, tutorial de classes, seção **9.5 Herança**:

[docs.python.org/pt-br/3/tutorial/classes.html](https://docs.python.org/pt-br/3/tutorial/classes.html)

A página explica o que acontece quando uma classe filha define um método com o mesmo nome de um método da mãe. Responda com suas palavras: quando você chama `bicho.agir(heroi)` e `bicho` é um `Goblin`, quem o Python procura primeiro?

---

E responda também: o `Orc`, que ainda não sobrescreveu `agir`, continua funcionando. Por quê?

---

# Decidir sem inventar estado

## O problema

### O PROBLEMA

O Goblin agora tem personalidade. Mas o Orc continua sendo um Goblin com números maiores, e o Troll também. Se você quiser dar comportamento a eles, a tentação vai ser criar contadores: um `self._turnos_de_furia` no Orc, um `self._ja_regenerou` no Troll, e por aí vai.

Cada um desses contadores é um pedacinho de estado que alguém vai precisar zerar, atualizar e lembrar que existe. Faça isso em cinco classes e você tem um jogo que ninguém consegue mais alterar sem quebrar.

## A explicação

Existe uma saída, e ela é quase sempre a mesma em programação: **antes de criar, procure o que já existe.**

O seu jogo já sabe fazer um monte de coisa. Faça o inventário mental:

| O QUE O JOGO JÁ SABE FAZER                    | QUEM FAZ                                        |
|-----------------------------------------------|-------------------------------------------------|
| Causar dano respeitando defesa                | <code>atacar</code> e <code>receber_dano</code> |
| Reduzir o dano do próximo golpe               | <code>defender</code>                           |
| Recuperar vida sem passar do máximo           | <code>curar</code>                              |
| Aumentar o ataque por alguns turnos           | <code>aplicar_bonus_ataque_temporario</code>    |
| Aumentar a defesa por alguns turnos           | <code>aplicar_bonus_defesa_temporario</code>    |
| Fazer o tempo passar                          | <code>iniciar_turno</code>                      |
| Guardar quanto falta para a habilidade voltar | <code>_recarga_habilidade</code>                |

Olhe as duas linhas dos bônus temporários. Você escreveu esses métodos na aula de efeitos temporários e, desde então, só os **itens** os usaram: o Elixir de Fúria e o Tônico de Pedra. O jogador bebe, o número sobe, e depois de três turnos volta ao normal sozinho.

Agora repare no que isso significa. Aquele sistema não pertence aos itens. Ele pertence ao `Personagem`. E **todo inimigo é um Personagem**.

#### A DESCOBERTA DO CAPÍTULO

Um Orc que se enfurece não precisa de um sistema de fúria. Ele precisa chamar `self.aplicar_bonus_ataque_temporario(6, 3)` — o mesmo método que o Elixir de Fúria chama. O contador, a duração e a hora de desfazer já estão implementados, testados e rodando desde a semana passada.

Isso é reúso de verdade, e é a diferença entre um sistema e uma pilha de casos especiais. Quando você escreve algo bem encapsulado, ele fica disponível para usos que você nem tinha imaginado quando escreveu.

#### UMA ARMADILHA: DECISÃO QUE SE REPETE TODO TURNO

Cuidado com um detalhe. Se o Orc decidir assim:

```
def agir(self, alvo):
    if self.vida < self.vida_maxima * 0.5:
        self.aplicar_bonus_ataque_temporario(6, 3)
    self.atacar(alvo)
```

ele vai se enfurecer **de novo em todo turno** em que estiver abaixo de metade da vida. O bônus nunca acaba, porque é renovado antes de expirar. Na prática o Orc vira um monstro permanente.

Você tem duas saídas honestas. A primeira é usar acaso, que o jogo já usa em `atacar`:

```
if self.vida < self.vida_maxima * 0.5 and random.randint(1, 4) == 1:
```

A segunda é fazer a fúria custar o turno — ele se enfurece **ou** ataca, nunca os dois:

```
if self.vida < self.vida_maxima * 0.5 and random.randint(1, 4) == 1:
    self.aplicar_bonus_ataque_temporario(6, 3)
    return
self.atacar(alvo)
```

A segunda é melhor de jogar, porque o inimigo paga um preço pela decisão e o jogador ganha uma janela para reagir. Decisão sem custo não é decisão.

### EXEMPLO 1 · O CARRO E O SISTEMA QUE JÁ EXISTIA

Um carro tem um sistema de freio, escrito uma vez, que sabe frear com a força que pedirem:

```
class Carro:
    def __init__(self):
        self.velocidade = 0

    def frear(self, forca):
        # o sistema de freio: um lugar só
        self.velocidade = max(0, self.velocidade - forca)
        print("Freando com forca", forca)

    def acelerar(self):
        self.velocidade += 10
```

Quando chega o piloto automático, ele **não** escreve um segundo sistema de freio:

```
class CarroAutonomo(Carro):
    def dirigir(self, distancia_do_obstaculo):
        if distancia_do_obstaculo < 5:
            self.frear(30)      # reusa o freio que já existia
        elif distancia_do_obstaculo < 20:
            self.frear(10)
        else:
            self.acelerar()
```

O piloto automático é **só decisão**. Ele escolhe quando frear e com que força, e quem sabe frear continua sendo o carro. Se o freio for melhorado amanhã, o piloto automático melhora junto sem uma linha alterada.

## EXEMPLO 2 · O MESMO MÉTODO, DOIS USUÁRIOS DIFERENTES

Este exemplo é o coração do capítulo. Uma conta bancária tem um método de bloqueio temporário:

```
class Conta:
    def __init__(self, dono):
        self.dono = dono
        self._dias_bloqueada = 0

    def bloquear_por(self, dias):
        self._dias_bloqueada = dias
        print("Conta de", self.dono, "bloqueada por", dias, "dias")

    def passar_dia(self):
        if self._dias_bloqueada > 0:
            self._dias_bloqueada -= 1
```

Quem chama `bloquear_por`? No começo, só o gerente. Depois chega o sistema antifraude, que é outra coisa completamente diferente — e usa o **mesmo** método:

```
class Antifraude:
    def analisar(self, conta, valor):
        if valor > 50000:
            conta.bloquear_por(2)  # o mesmo metodo do gerente
```

Ninguém escreveu um segundo sistema de bloqueio para o antifraude. A conta sabe se bloquear; quem decide pode ser qualquer um.

Troque `Conta` por `Personagem`, `bloquear_por` por `aplicar_bonus_ataque_temporario`, o gerente por `ElixirDeFuria` e `Antifraude` pelo `Orc`. É exatamente a mesma figura.

### EXERCÍCIO 3 · O ORC QUE SE ENFURECE

ARQUIVO: CLASSES.PY

Faça:

1. Na classe `Orc`, sobrescreva `agir`.
2. Quando ele estiver abaixo de metade da vida, ele pode entrar em fúria: chame o método de bônus de ataque temporário que já existe, escolhendo a intensidade e a duração.
3. Garanta que a fúria **não** se renove todo turno. Use uma das duas saídas da explicação, ou invente a sua e justifique.
4. Se ele entrou em fúria, esse é o turno dele: não ataque também.
5. Em qualquer outro caso, ele ataca normalmente.

Você vai precisar do `random`, que o `classes.py` já importa lá no topo.

### EXERCÍCIO 4 · O TROLL QUE SE PROTEGE

ARQUIVO: CLASSES.PY

O Troll já sobrescreve `atacar`: ele erra muito, mas quando acerta dói. Isso **continua igual** — você não vai encostar no `atacar` dele.

Faça:

1. Sobrescreva `agir` no `Troll`.
2. Abaixo de 40% da vida, ele se protege usando o método de bônus de **defesa** temporário que já existe.
3. Aplique a mesma cautela do exercício 3 para não renovar o bônus todo turno.
4. Em qualquer outro caso, ele ataca — e, quando atacar, tem que continuar sendo o ataque desastrado dele, com a chance de errar feio.

**Confira depois de escrever:** o seu `agir` chama `self.atacar(alvo)`. Se você tiver escrito `Personagem.atacar(self, alvo)`, o Troll perde a identidade que ele tinha desde a aula de herança.

#### TESTE DOS EXERCÍCIOS 3 E 4

```
from classes import Orc, Guerreiro

heroi = Guerreiro("Thoric")

orc = Orc()
orc._vida = 20          # abaixo da metade de 70
ataque_antes = orc.ataque

for _ in range(8):
    orc.iniciar_turno()
    orc.agir(heroi)

print("ataque antes:", ataque_antes, "| agora:", orc.ataque)
```

**Resultado esperado:** em algum momento aparece a mensagem de fúria e o ataque sobe. Depois de alguns turnos aparece a mensagem de que o ataque voltou ao normal, e no fim o ataque é **igual** ao de antes.

**Se o ataque final ficou maior que o inicial:** a fúria está sendo renovada antes de expirar. É exatamente a armadilha da explicação.

**Se nada aconteceu em 8 turnos:** pode ser azar do `random` — rode de novo. Se nunca acontecer, confira a comparação da vida.

**Repare no `iniciar_turno` dentro do laço.** Sem ele o tempo não passa e o bônus nunca expira. No jogo de verdade, quem chama isso é o combate.

**Uma pergunta para levar para amanhã.** Conte quantos atributos o `Personagem` gasta hoje para sustentar os bônus temporários: intensidade e duração do ataque, intensidade e duração da defesa. São quatro. Some a recarga da habilidade e vão cinco. Agora conte quantos blocos parecidos existem dentro do `iniciar_turno`.

Não mexa em nada disso hoje. Só anote o número. Ele vai ser o assunto da próxima aula.

# O contrato de ontem, do outro lado

## O problema

Abra o `Dragao` no seu `classes.py`. Ele tem um sopro de fogo, e o sopro está escrito dentro do `atacar`, mais ou menos assim:

```
class Dragao(Personagem):
    def __init__(self):
        super().__init__("Dragão Ancião", 110, 14, 4, 0)
        self._turnos = 0

    def atacar(self, alvo):
        self._turnos += 1
        if self._turnos % 3 == 0:
            # o sopro mora aqui dentro
            ...
        super().atacar(alvo)
```

### O PROBLEMA

Três coisas estão erradas nesse desenho, e todas as três você já sabe consertar.

**Um:** o sopro é uma habilidade especial, mas não se chama `habilidade_especial`. O jogo tem um contrato para isso desde ontem, e o Dragão o ignora.

**Dois:** `self._turnos` é um contador de recarga com outro nome. O `Personagem` já tem `_recarga_habilidade`, que já é atualizado dentro de `iniciar_turno`. São dois relógios para o mesmo trabalho.

**Três:** o `atacar` do Dragão decide e executa ao mesmo tempo — é exatamente a confusão do capítulo 1, ainda de pé no meio do seu código.

## A explicação

Ontem você escreveu, na classe `Personagem`, três coisas: o método `habilidade_especial(self, alvo)`, o atributo `_recarga_habilidade` e a linha dentro de `iniciar_turno` que faz esse contador descer.

Naquele momento tudo isso existia para o **herói**. Você testou com Guerreiro e Mago, ligou no menu, mostrou no status.

Mas olhe onde você escreveu. Não foi no Guerreiro. Foi no `Personagem`.

### A CONSEQUÊNCIA

Goblin, Orc, Troll, Esqueleto e Dragão **já têm** `habilidade_especial` e `_recarga_habilidade`. Eles herdaram tudo ontem, no instante em que você salvou o arquivo. Nunca usaram porque ninguém chamou.

Você não vai construir um sistema de habilidade para inimigo. Você vai **ligar** um que já está pronto e já está rodando dentro deles.

Esse é o momento em que o desenho de ontem se paga. Quando o contrato mora na classe base, ele vale para toda a família — inclusive para os membros da família que você nem estava pensando quando escreveu.

### COMO O `AGIR` CONSULTA A RECARGA

A decisão do inimigo fica com esta forma:

```
def agir(self, alvo):
    if self._recarga_habilidade == 0:
        self.habilidade_especial(alvo)
    else:
        self.atacar(alvo)
```

Leia devagar, porque tem sutileza aqui. O `agir` pergunta se a habilidade está pronta, mas **não** mexe no contador. Quem zera, quem inicia a espera e quem imprime "ainda faltam 2 turnos" é o `habilidade_especial`, exatamente como você escreveu ontem. E quem faz o tempo passar continua sendo o `iniciar_turno`.

Três responsabilidades, três lugares, nenhum deles duplicado:

| QUEM                             | RESPONSABILIDADE                        |
|----------------------------------|-----------------------------------------|
| <code>iniciar_turno</code>       | Fazer o tempo passar.                   |
| <code>habilidade_especial</code> | Verificar, executar e iniciar a espera. |
| <code>agir</code>                | Só decidir se vale a pena tentar.       |

**E se o `agir` nem consultasse a recarga?** Também funcionaria: ele chamaria `habilidade_especial` sempre, e o método recusaria educadamente quando estivesse em recarga. Mas aí o inimigo **perderia o turno** tentando usar algo indisponível — que é justamente a regra que o material de ontem definiu para o jogador. Para o inimigo isso é burrice, não dificuldade. Por isso ele pergunta antes.

### EXEMPLO 1 · A HABILIDADE HERDADA QUE NINGUÉM USAVA

Uma família de veículos, onde a base define um recurso de emergência:

```
class Veiculo:
    def buzinar(self):
        print("Bi-bi")

    def acionar_emergencia(self):
        print("Pisca-alerta ligado")

    def andar(self):
        print("Andando")

class Ambulancia(Veiculo):
    def acionar_emergencia(self):
        print("Sirene e giroflex ligados")
```

Escrito assim, `acionar_emergencia` parece coisa de ambulância. Mas um dia o sistema de sensores chega e faz isto:

```
for veiculo in garagem:
    if veiculo.detectou_colisao():
        veiculo.acionar_emergencia()
```

E todo carro comum da garagem passa a ligar o pisca-alerta sozinho — sem nenhuma linha nova em `Veiculo`. O recurso estava lá desde sempre, esperando alguém chamar.

É a situação exata do Dragão com o `habilidade_especial` herdado ontem.

## EXEMPLO 2 · VERIFICAR ANTES DE AGIR, COM O CONTADOR DE TERCEIROS

Retome a cafeteira que apareceu no material de ontem, agora com um dono que decide:

```
class Cafeteira:
    def __init__(self):
        self._espera = 0

    def preparar(self):
        if self._espera > 0:
            print("Ainda aquecendo, faltam", self._espera)
            return
        print("Cafe pronto")
        self._espera = 2

    def servir_agua(self):
        print("Copo de agua")

    def passar_minuto(self):
        if self._espera > 0:
            self._espera -= 1
```

```
class Recepcao:
    def __init__(self):
        self.cafeteira = Cafeteira()

    def atender(self):
        # decide: so tenta o cafe se valer a pena
        if self.cafeteira._espera == 0:
            self.cafeteira.preparar()
        else:
            self.cafeteira.servir_agua()

    def passar_minuto(self):
        self.cafeteira.passar_minuto()
```

Três coisas para reparar, e as três reaparecem no seu Dragão:

- A recepção **consulta** a espera, mas nunca a altera. Quem altera é a cafeteira.
- Quando não dá café, a recepção não fica parada: ela faz outra coisa útil.
- O relógio corre num lugar só, no `passar_minuto`. Ninguém tem um segundo contador.

## EXERCÍCIO 5 · O SOPRO VIRA HABILIDADE ESPECIAL

ARQUIVO: CLASSES.PY

Faça:

1. No `Dragao`, crie `habilidade_especial(self, alvo)`, com a mesma assinatura que o `Guerreiro` e o `Mago` usam.
2. Mova o sopro de fogo para dentro dele. O sopro continua atravessando a defesa do alvo, e continua sem tocar em `_vida`.
3. Aplique a mesma regra de recarga que você usou no herói: verificar antes, iniciar a espera só depois de um uso bem-sucedido.
4. Devolva o `atacar` do Dragão ao estado normal: ele volta a ser só um ataque, sem `if` de sopro e sem contador.
5. Apague o atributo `self._turnos` do `__init__` dele. O `_recarga_habilidade` faz esse trabalho.
6. Sobrescreva `agir`: se a habilidade estiver disponível, use; senão, ataque.

**Escolha e anote:** qual é o tamanho da recarga do Dragão? Ele é o chefe final. Se a recarga for curta demais, o combate vira injústo; longa demais, ele vira inofensivo. Jogue e ajuste.

### TESTE DO EXERCÍCIO 5

```
from classes import Dragao, Guerreiro

heroi = Guerreiro("Thoric")
chefe = Dragao()

for turno in range(1, 8):
    chefe.iniciar_turno()
    print("--- turno", turno, "| recarga:", chefe._recarga_habilidade)
    chefe.agir(heroi)

print("vida do heroi:", heroi.vida)
```

**Resultado esperado:** no turno 1 o sopro sai. Nos turnos seguintes o Dragão ataca normalmente, enquanto o contador desce. Quando o contador chega a zero, o sopro volta. O padrão se repete.

**Se o sopro sair em todo turno:** o `habilidade_especial` não está iniciando a recarga depois do uso.

**Se o sopro nunca mais voltar:** o contador não está descendo. Confira se a linha que você escreveu ontem dentro de `iniciar_turno` continua lá, e se o teste está chamando `iniciar_turno` a cada volta.

**Se der `AttributeError` sobre `_turnos`:** sobrou alguma referência ao atributo antigo no `atacar`. Ele deve ter sumido junto.

## EXERCÍCIO 6 · O ESQUELETO ENTRA EM CAMPO

ARQUIVO: CLASSES.PY, DEPOIS MAIN.PY

Existe uma classe `Esqueleto` no seu `classes.py` que nunca entrou em nenhuma fase. Hoje ela entra.

Faça:

1. Dê ao `Esqueleto` uma `habilidade_especial` própria, que não seja cópia do sopro nem do golpe do Guerreiro. Ele é um morto que se remonta — algo ligado a recuperar-se combina.
2. Escolha uma recarga coerente com a força dessa habilidade.
3. Sobrescreva `agir` para que ele só use a habilidade quando ela fizer sentido. Curar-se com vida cheia é desperdício: some uma condição de vida à condição de recarga.
4. No `main.py`, acrescente uma fase nova com o Esqueleto, com nome, descrição e prêmio.

**A restrição do dia continua valendo:** nenhum atributo novo. Tudo que você precisa — vida, vida máxima, recarga — já está lá.

### O TESTE QUE VALE MAIS QUE OS OUTROS

Depois de acrescentar a fase do Esqueleto, abra o `main.py` e procure a função do combate.

#### Quantas linhas você precisou mudar nela para o Esqueleto funcionar?

A resposta certa é **zero**. O combate chama `inimigo.agir(jogador)`, e o Esqueleto respondeu por conta própria. Você acrescentou um inimigo com comportamento inédito sem que o combate ficasse sabendo que ele existe.

Se você precisou mexer no combate, tem um `if` sobre tipo escondido em algum lugar. Ache e tire.

# A prova do desenho

## O problema

### O PROBLEMA

Todo mundo acha que o próprio código está bem organizado. A sensação não vale nada. O que vale é medir.

Existe uma medida honesta para o que você fez hoje, e ela é incômoda porque é objetiva:  
**quantos arquivos você precisa abrir para acrescentar um inimigo novo?**

## A explicação

Um desenho bom não é o que parece bonito. É o que faz a mudança seguinte ficar barata.

De manhã, para dar um comportamento diferente a um inimigo, você teria que abrir o `main.py` e escrever alguma coisa parecida com isto:

```
if isinstance(inimigo, Goblin):
    if inimigo.vida < 12:
        inimigo.defender()
    else:
        inimigo.atacar(jogador)
elif isinstance(inimigo, Orc):
    ...
elif isinstance(inimigo, Dragao):
    ...
```

Esse trecho tem um nome informal na indústria: **escadinha**. E ele tem três defeitos que valem mais que o incômodo visual:

- **Cresce para sempre.** Cada inimigo novo é mais um `elif`. A função nunca fica pronta.
- **Fica longe de quem sabe.** A regra do Goblin está no `main.py`, mas os números do Goblin estão no `classes.py`. Para entender o bicho você lê dois arquivos.
- **Espalha o erro.** Se existir uma segunda escadinha em outro lugar — na hora de mostrar o status, por exemplo — você precisa lembrar de atualizar as duas.

A versão de agora não tem nada disso. O combate diz `inimigo.agir(jogador)` e cala a boca. Cada regra mora ao lado dos números da própria classe.

#### A FRASE PARA LEVAR

Quando você precisa perguntar **que tipo** um objeto é para saber o que fazer com ele, quase sempre a resposta certa é: pare de perguntar e deixe o objeto responder. Isso é polimorfismo, e é a coisa mais útil que a POO te dá.

#### EXEMPLO 1 · A MESMA ESCADINHA, NO MUNDO REAL

Um sistema de pagamentos escrito com escadinha:

```
def cobrar(pagamento, valor):
    if pagamento.tipo == "cartao":
        print("Cobrando", valor + 2, "no cartao")
    elif pagamento.tipo == "boleto":
        print("Gerando boleto de", valor)
    elif pagamento.tipo == "pix":
        print("Pix de", valor)
```

O mesmo sistema com polimorfismo:

```
class Pagamento:
    def cobrar(self, valor):
        print("Forma de pagamento nao configurada")

class Cartao(Pagamento):
    def cobrar(self, valor):
        print("Cobrando", valor + 2, "no cartao")

class Pix(Pagamento):
    def cobrar(self, valor):
        print("Pix de", valor)
```

Agora acrescente uma forma de pagamento nova em cada versão e conte os arquivos que você abre. Na primeira, você abre a função `cobrar` — e todas as outras que também têm escadinha sobre `tipo`. Na segunda, você escreve uma classe e acabou.

## EXEMPLO 2 · O TESTE DO ARQUIVO ÚNICO

Um jeito rápido de avaliar qualquer desenho orientado a objetos, que você pode aplicar pelo resto da vida:

PERGUNTA: para acrescentar uma coisa nova do mesmo tipo  
(um inimigo, um item, uma forma de pagamento),  
quantos arquivos eu preciso abrir?

1 arquivo -> o desenho está certo  
2 arquivos -> aceitável se o segundo for só cadastro  
3 ou mais -> tem escadinha escondida em algum lugar

Aplique ao seu jogo hoje. Para acrescentar um inimigo com comportamento próprio, você abre o `classes.py` para escrever a classe e o `main.py` para cadastrar a fase. Dois arquivos, e o segundo é só cadastro.

Compare com o item: você já passou por isso na aula de composição, e a conta deu o mesmo. Não é coincidência. É o mesmo desenho aplicado duas vezes.

## EXERCÍCIO 7 · PROVAR QUE O DESENHO AGUENTA

ARQUIVO: `CLASSES.PY`, DEPOIS `MAIN.PY`

Invente um inimigo que ainda não existe no jogo. Ele precisa ter um comportamento que **nenhum** dos outros tem — não vale copiar o Goblin com outros números.

Algumas ideias, se você travar:

- Um bicho que fica **mais forte** quanto mais ferido está.
- Um inimigo que se cura quando o jogador está se defendendo, porque sabe que não vai levar dano.
- Um bicho que ataca duas vezes seguidas quando o jogador está com pouca vida.
- Um inimigo que na primeira metade da luta só se defende, e depois parte para cima.

Faça:

1. Escreva a classe no `classes.py`, herdando de `Personagem`.
2. Sobrescreva `agir` com a decisão dele. Sobrescreva `habilidade_especial` se fizer sentido.
3. Cadastre uma fase nova no `main.py`.
4. **Antes de rodar**, escreva num papel quantas linhas do combate você acha que vai precisar mudar.
5. Rode. Compare com o que você escreveu.

## O que você construiu hoje

| CONCEITO                   | ONDE ESTÁ NO SEU JOGO                                                                                               |
|----------------------------|---------------------------------------------------------------------------------------------------------------------|
| Decisão separada de ação   | <code>agir</code> escolhe; <code>atacar</code> , <code>defender</code> e <code>habilidade_especial</code> executam. |
| Polimorfismo sobre decisão | Cada inimigo sobrescreve <code>agir</code> ; o combate nunca pergunta o tipo.                                       |
| Reúso de sistema pronto    | Orc e Troll usam os bônus temporários escritos para os itens.                                                       |
| Contrato na classe base    | <code>habilidade_especial</code> e <code>_recarga_habilidade</code> valiam para o inimigo desde ontem.              |
| Encapsulamento             | A decisão consulta a recarga; só o dono a altera.                                                                   |

### UMA ÚLTIMA OBSERVAÇÃO, E ELA É PARA AMANHÃ

Hoje você não escreveu quase nenhuma ação nova. Escreveu **critérios**. E mesmo assim o jogo mudou mais do que em qualquer outra aula desde que ele virou jogo.

Isso aconteceu porque as peças já estavam prontas e bem separadas. Repare, porém, no preço que essas peças cobram hoje: cinco atributos de controle dentro do `Personagem`, três blocos parecidos no `iniciar_turno`, e um método `aplicar_bonus...` para cada tipo de bônus. Funciona — e está começando a pesar.

Amanhã a gente mexe nisso.

# Erros que você vai ver

## 1. RECURSIONERROR: MAXIMUM RECURSION DEPTH EXCEEDED

```
RecursionError: maximum recursion depth exceeded
```

Você escreveu `agir` chamando `agir`. Acontece muito quando se copia o método de uma classe para outra e esquece de trocar o nome de dentro.

### ERRADO

```
def agir(self, alvo):  
    self.agir(alvo)
```

### CERTO

```
def agir(self, alvo):  
    self.atacar(alvo)
```

## 2. O INIMIGO VIROU GENÉRICO: O TROLL PERDEU O GOLPE DESASTRADO

Sem erro na tela. O Troll simplesmente ataca como qualquer um.

O seu `agir` está chamando o ataque pela classe, e não pelo objeto. Chamar pela classe ignora a sobrescrita.

### ERRADO

```
Personagem.atacar(self, alvo)
```

### CERTO

```
self.atacar(alvo)
```

## 3. TYPEERROR: AGIR() TAKES 1 POSITIONAL ARGUMENT BUT 2 WERE GIVEN

```
TypeError: agir() takes 1 positional argument but 2 were given
```

Você escreveu `def agir(self):` sem o `alvo`, mas o combate chama `inimigo.agir(jogador)`. A assinatura tem que ser igual em todas as classes — é isso que faz o polimorfismo funcionar.

#### 4. O BÔNUS TEMPORÁRIO NUNCA ACABA

Sem erro na tela. O ataque do Orc sobe e nunca mais desce.

Duas causas possíveis. A primeira: a decisão está renovando o bônus todo turno, antes de ele expirar — é a armadilha do capítulo 3. A segunda: o `iniciar_turno` do inimigo não está sendo chamado no combate, então o tempo não passa para ele.

Confira no seu `main.py` se existe uma chamada de `iniciar_turno` para o inimigo, e não só para o jogador.

#### 5. A HABILIDADE DO INIMIGO SAI EM TODO TURNO

O Dragão sopra fogo sem parar.

O `habilidade_especial` dele executa mas não inicia a recarga. Compare com o que você escreveu ontem no Guerreiro: lá a espera começa **depois** do sucesso, dentro do próprio método.

Outra possibilidade: você iniciou a recarga dentro do `agir` em vez do `habilidade_especial`. Aí o herói, que chama a habilidade pelo menu, ficaria sem recarga nenhuma.

#### 6. ATTRIBUTEERROR: OBJECT HAS NO ATTRIBUTE '\_TURNOS'

```
AttributeError: 'Dragao' object has no attribute '_turnos'
```

Sobrou uma referência ao contador antigo do Dragão em algum lugar depois do exercício 5, ou você copiou o `agir` do Dragão para outra classe sem adaptar.

#### 7. O INIMIGO SE DEFENDE MAS CONTINUA TOMANDO DANO CHEIO

Sem erro na tela. O Goblin recua, e o golpe seguinte dói igual.

Confira a ordem das coisas no combate. O `iniciar_turno` zera o `defendendo`. Se ele for chamado entre a defesa do inimigo e o ataque do jogador, a defesa é apagada antes de servir para alguma coisa.

No arranjo correto, o inimigo se defende no fim do turno dele e essa defesa vale contra o ataque do jogador no turno seguinte.

# Extras

---

Tudo aqui é opcional. Nenhum capítulo depende destes itens.

## E1.1 O INIMIGO QUE FALA O QUE VAI FAZER

Depende de: exercício 1.

Faça o `agir` do `Personagem` imprimir uma linha antes de executar, dizendo o que o inimigo escolheu.

Depois responda: isso deveria estar no `agir` ou dentro de cada ação? Existe um argumento bom para cada lado — escolha um e defenda.

## E2.1 O INIMIGO QUE BEBE POÇÃO

Depende de: exercício 2.

Dê duas poções a um inimigo no `__init__` dele e faça o `agir` usar uma quando estiver ferido, em vez de se defender.

Você vai precisar da mochila da aula de composição: o inimigo tem um `inventario` desde então. Use `tirar` e `usar`, exatamente como o menu do jogador faz.

Repare no que aconteceu: o inimigo passou a usar o sistema de itens sem que o sistema de itens soubesse que inimigos existem.

## E2.2 OLHAR PARA O ADVERSÁRIO

Depende de: exercício 2.

Até agora as decisões só leram o próprio estado. Mas o `agir` recebe `alvo`, e o alvo também tem vida.

Faça um inimigo que arrisca tudo quando percebe o jogador quase morto: abaixo de 20% de vida do alvo, ele ataca mesmo estando ferido, sem se defender.

Responda: por que ler `alvo.vida` é seguro, mas escrever `alvo._vida` não seria?

### E3.1 A FÚRIA COM CUSTO VISÍVEL

Depende de: exercício 3.

Faça a fúria do Orc custar vida: ele ganha ataque, mas perde uma parte da vida ao se enfurecer.

Use `receber_dano` ou mexa direto? Pense: `receber_dano` vai descontar a defesa dele, o que provavelmente anula o custo. Decida como resolver isso sem quebrar o encapsulamento, e escreva a sua justificativa.

### E4.1 DUAS HABILIDADES, UMA ESCOLHA

Depende de: exercício 5.

Dê ao Dragão uma segunda habilidade e faça o `agir` escolher entre as duas conforme a situação — o sopro quando o jogador está com vida alta, outra coisa quando está quase morto.

Pergunta difícil: as duas habilidades devem dividir a mesma recarga ou ter recargas separadas? As duas respostas são defensáveis. Escolha, implemente, e explique o efeito da sua escolha no equilíbrio da luta.

### E4.2 O JOGADOR VÊ A INTENÇÃO

Depende de: exercício 5.

Acrescente ao `mostrar_status` do inimigo a informação de recarga que você já mostra para o jogador.

Isso muda o jogo: agora o jogador sabe que o sopro vem no próximo turno e pode se defender. Jogue algumas partidas e decida se ficou melhor ou pior. Não existe resposta certa — existe a sua, justificada.

### E5.1 LEITURA DIFÍCIL

Depende de: exercício 7.

Este extra é de leitura. Abra a documentação das funções embutidas:

[docs.python.org/pt-br/3/library/functions.html#isinstance](https://docs.python.org/pt-br/3/library/functions.html#isinstance)

O material desta aula manda evitar `isinstance` no combate. Mas a função existe e é útil em algum lugar, senão não estaria na biblioteca padrão.

Responda: cite uma situação em que usar `isinstance` é a escolha certa, e explique por que ela é diferente do caso do combate.

# Checklist de entrega

---

## FUNCIONA

- ☐ O jogo roda do início ao fim sem erro na tela.
  - ☐ O Goblin se defende quando está quase morto.
  - ☐ O Orc entra em fúria e a fúria acaba sozinha depois de alguns turnos.
  - ☐ O Troll se protege quando ferido e continua errando feio quando ataca.
  - ☐ O Dragão alterna entre sopro e ataque comum, com a recarga funcionando.
  - ☐ O Esqueleto entrou numa fase e faz alguma coisa própria.
- 

## DECISÃO SEPARADA DE AÇÃO

- ☐ `Personagem` tem `agir`, e a versão da mãe só ataca.
  - ☐ Todo `agir` chama ações por `self`, nunca por `Personagem`.
  - ☐ Nenhum `agir` calcula dano nem imprime resultado de ataque.
  - ☐ Todos os `agir` têm a mesma assinatura: `self` e `alvo`.
- 

## O COMBATE NÃO CONHECE CLASSES

- ☐ O `main.py` não contém `isinstance`.
  - ☐ O `main.py` não contém `type(`.
  - ☐ O combate chama `inimigo.agir(jogador)` e nada mais.
  - ☐ Acrescentar um inimigo novo não exigiu tocar na função de combate.
- 

## NENHUM ESTADO INVENTADO

- ☐ Nenhuma classe de inimigo ganhou atributo novo nos capítulos 2 e 3.
  - ☐ O `self._turnos` do Dragão foi apagado.
  - ☐ A fúria do Orc usa `aplicar_bonus_ataque_temporario`.
  - ☐ A proteção do Troll usa `aplicar_bonus_defesa_temporario`.
  - ☐ Quem faz o tempo passar continua sendo só o `iniciar_turno`.
-

## ENCAPSULAMENTO

☐ Nenhum `agir` escreve em `_vida`, nem no próprio nem no do alvo.

---

☐ Nenhum `agir` escreve em `_recarga_habilidade`.

---

☐ As decisões usam proporção de vida, não números fixos.

---

### O ÚLTIMO TESTE

Escolha um inimigo qualquer e mude uma regra dele — o limite de vida em que ele se protege, por exemplo. Conte quantos arquivos você abriu.

Se a resposta for "um", o comportamento está onde deveria estar: dentro do bicho.